MLVU information sheet

Please include this page in your report either at the start or at the end, before the appendix. Do not change the formatting.

Group number

156

Authors

name	student number
Conrad Clough	2699315
Samuel Power	2800107
Andreas Stolle	2808634

Software used *Describe briefly which software and libraries you relied on. If you built a particular algorithm from scratch, make sure to state it explicitly here.*

We used Jupyter, numpy, pandas, matplotlib, and sklearn. Jupyter was used for ease of modifying our program without needing to redo data processing; numpy, pandas, and matplotlib were used for interacting with data and displaying results; sklearn was used for their model implementations.

Use of AI tools *If you used tools like ChatGPT or Github Copilot in any phase of the report, please detail here which tools you used, what parts of the work you used them for and how. Note that if you use an AI tool and don't report it, it's considered fraud, so be detailed. Note also that having an AI tool write for you is not allowed. If no AI tools were used, please state that explicitly.*

- Conrad: ChatGPT was used to generate the code that created our graphs, specifically the styling.

Link to code (optional) https://github.com/Flootersmm/ml_project

Group disagreements (optional) *If there are any disagreements in the group, you may **not** remove a student from the author list without their consent. You should bring disagreements to our attention early. As a last resort, you can describe any grievances here.*

How can machine learning-based DDoS detection models be designed to minimise false positive rates while maintaining high detection accuracy on both low- and high-powered hardware?

Conrad Clough
VUNetId: cch250
Student Nr: 2699315
c.j.clough@student.vu.nl

Samuel Power
VUNetId: spo237
Student Nr: 2800107
s.j.power@student.vu.nl

Andreas Stolle
VUNetId: ast451
Student Nr: 2808634
a.stolle@student.vu.nl

I. Abstract

Distributed Denial of Service (DDoS) attacks continue to disrupt network services, causing downtime and economic damage. Existing techniques for mitigating this often struggle to deal with evolving attack signatures [1]. This paper investigates the design of DDoS detection models using machine learning. We evaluate three classifiers, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forests, by analysing their accuracy, false positive rates, and resource usage under controlled conditions. Our results indicate that kNN is too simple of a model to efficiently classify attacks, RF is significantly faster and very usable in all levels of hardware but slow to train, and LR beats the previous two models in every meaningful way. These findings suggest that model selection is dependent on hardware specifications and the relative importance of minimising false positives, providing a practical guide for using machine learning-based DDoS detection in diverse contexts, but future work should be done to identify areas where LR may perform suboptimally.

Keywords: DDoS detection, machine learning, false positive rate, k-Nearest Neighbours, Random Forest, Logistic Regression, computational efficiency

II. Introduction

The proliferation of online services has made networks increasingly vulnerable to Distributed Denial of Service (DDoS) attacks, which can overwhelm servers and disrupt them, causing significant operational downtime and economic damage. Detecting these attacks efficiently is challenging because high accuracy must be balanced with low false positive rates; denying service to legitimate users is highly undesirable, as is allowing too much malicious traffic through. Machine learning offers a promising alternative to traditional, rule-based approaches; patterns in network traffic can be used to identify evolving malicious activity in real time. This document investigates the design of machine learning-based DDoS detection models with an emphasis on maintaining high detection accuracy while reducing false positives, and evaluates their resource usage across a range of hardware to ensure usability for a variety of situations. Specifically, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forest classifiers are compared in terms of both accuracy, false positive rate, and performance.

The report is structured as follows:

- In Section III we consider existing methods of DDoS detection.
- In Section IV we discuss the dataset we used and how it was prepared for use in training.
- In Section V we explain our classifiers and how we ran our experiments.
- In Section VI we show and discuss our results.
- In Section VII we summarise our findings.

III. Related Work

Previously, rule-based classifiers were used to detect DDoS attacks [1]. This involves the categorisation data with a series of "if … then …" rules, generating descriptive models that are easy to interpret. This is an earlier machine learning method than the classifiers explored in this report, and therefore less expressive. Requiring distinct rules that must be carefully considered to provide good coverage and classification makes this kind of model perform poorly in exceptional scenarios. Real-world data is often fuzzy, and preventing DDoS attacks is an arms race - the complexity and upkeep of a rule-based classifier fighting new attacks with evolving signatures makes them unwieldy.

There are many ways to use kNN, and one notable study reduces the feature space drastically to only four features [2]. In constrast, this report applies kNn to all numeric features available in the dataset, a total of 79, to evaluate whether broader feature coverage improves performance or not.

Other papers have investigated the usage of a kNN model to tackle DDoS attacks in real-time. It's correctly pointed out in [3] that machine learning is a rapidly developing field, and therefore repeated investigation into model performance is important and relevant to consider. The author expressed interest in investigating the computational resource requirements for this real-time protection; resource-constrained IoT environments were less performant than desired and required further optimisation to reach a high level of effectiveness.

While kNN does work well in many cases, it's downside is that it relies on heavy distance computations, which can slow it down for devices with limited resources. Therefore, we want to see if a simpler model could still get similar results. A natural choice for this is Logistic Regression, as it is often used as lightweight baseline model for network intrusion detection and it doesn't consume much resources. In his analysis of classic machine learning classifiers, Boadi shows that LR does score fairly high in accuracy AUC, but is outperformed by more complex models like Random Forest [4].

Even if we are aware that LR might not deliver the best results out of all the models, it is still worth including, as its trade-off between performance and resource consumption is attractive and there are ways to maximise results. Research shows that LR can be improved significantly with careful preprocessing, such as feature selection and balanced data [5]. This means that LR's effectiveness depends on how the data is set up just as much as it does on how complex the model itself is. One study that focuses on using LR in deployment emphasises that can be scaled further through techniques like parallelised training and automated hyperparameter tuning, allowing for competitive performance while keeping computational overead low [6].

This means that LR is not just a popular baseline, but is especially a lightweight and effective model that using the right approach can be tuned to successfully handle the trade-offs between accuracy, falso positive rate and computational efficiency. RF papers here please!!!

IV. Dataset and Preprocessing

Three models were trained on a dataset consisting of network traffic, balanced between DDoS traffic and benign traffic.

These models were kNN, logistic regression, and random forest. The data was extracted from public IDS datasets, produced in different years and with different techniques of generating traffic. There are 12 794 627 datapoints and 84 features. The models' inference abilities were then tested on an identically formed dataset that was unbalanced, with 20% DDoS traffic and 80% benign. Implementation of each model was provided by the `sklearn` library for Python.

In order to manage resource and time constraints, training and inference were limited to 200 000 labels. This data was then put into a `pandas` dataframe for efficiency and ease of management. Particular features that were either irrelevant or non-numeric were also dropped - both kNN and logistic regression require numeric features. An alternative way of tackling non-numeric features is to one-hot encode them, where categoric features are given an integer identifier. This massively increases the memory requirements of the model and was therefore decided against, with the exception of encoding the `Protocol` feature which was the only relevant categorical feature present. This resulted in 79 features.

The next step was cleaning; all features were represented with a 64-bit float, invalid numbers were cast to NaN for predictable management, and missing values were imputed using the median.

Next, the dataset was split into 60% training, 20% validation, and 20% test data. The training set was used for fit the model parameters, and was the largest to allow the models to effectively capture patterns in the data. The validation set was used for hyperparameter tuning; as the models haven't encountered the data before this is a suitable playground for adjustment. The final test set allows finalisation of ideal hyperparameters, and an additional verification that they are indeed ideal without any bias. Notable, this is not training on the test set. It's a sanity check, and if unexpected results

Finally, features were then scaled with a mean of 0 and a standard deviation of 1, which is z-score normalisation. This is because logistic regression is sensitive to feature magnitude as it uses gradient descent which can struggle when, for example, one feature takes values $[0, 1]$ and another takes $[2^{32}, 2^{64}]$. This prevents particular features completely dominating calculations and having inflated importance. It also allows faster convergence to the gradient's local minima. An alternative, explored in [2], is to hugely reduce the feature space - this was opted against this as makes a comparison between kNN and other models less direct. This scaler was applied to the training data, and then used to transform all data to make it homogeneous for the models. This is the last step to prepare the data.

Exact implementation details can be viewed at the codebase's git repository [7].

V. Methodology

A. kNN

The first model evaluated was kNN. This works by finding the k-nearest points in the training dataset and uses their class to predict the class of nearby datapoints. For 79 features, the feature space has a dimensionality of $\mathbb{R}^{79}$ which makes visualisation challenging, but the idea of 'nearby datapoints' generalises to high-dimension Euclidean spaces and therefore kNN is perfectly usable, though slow (Table I). Varying hyperparameters k were tested to maximise accuracy and minimise overfitting, see Fig. 1. Larger k can lead to overfitting and smaller k can lead to underfitting, so finding the balance was important.

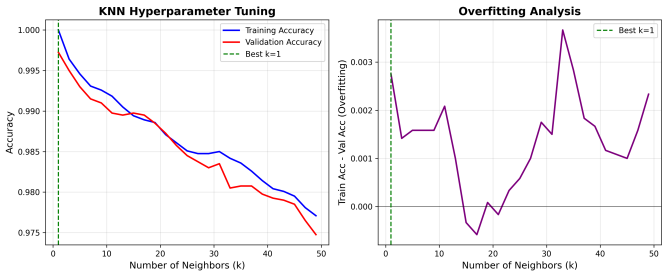

Fig. 1. Tuning hyperparameters for kNN for accuracy. $k = 1$ provides the best balance between accuracy and overfitting.

After the optimal hyperparameter k was chosen, 5-fold cross-validation was used, wherein the validation and training sets were combined and split into 5 sections. This allows for addressing the tradeoff between bias and variance, finding a model that maximises the benefit gained for the best inference results. Finally, the model was trained and then evaluated a single time on the test data. Using the test data a single time to evaluate final performance is extremely important to prevent provide an independent analysis of the model's performance - repeated use of the test data means this is no longer an evaluation on unseen data and therefore valueless.

B. Logistic Regression

For the logistic regression model, with an input feature vector x, it estimates the probability of the DDoS class as

$$P(y = 1 \mid x) = \sigma(w^T x + b), \tag{1}$$

where w is the coefficient vector, b is the bias term, and $\sigma(.)$ is the sigmoid function. Classification is obtained by thresholding this probability.

To study the sensitivity of logistic regression to L2 regularisation and to reduce overfitting, we evaluate C over a logarithmic range from 10^{-4} to 10^{4}. For each value of C, we measure training accuracy, validation accuracy, and the overfitting gap.

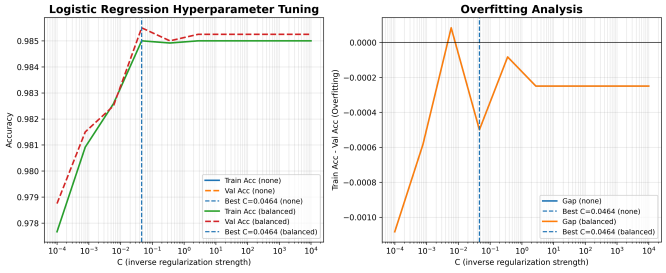

Fig. 2. Effect of the inverse regularisation parameter C on performance. Accuracy improves sharply at low C and quickly plateaus, while the overfitting gap remains negligible.

TABLE I
RESOURCE USAGE FOR MODEL TRAINING, WITH 200 000 SAMPLES.

Model	RAM Δ (MB)	CPU Time (s)	Wall Time (s)
kNN	2034.49	10121.29	1163.05
RF	2358.29	1619.48	116.39
LR	233.83	69.41	42.31

The best validation performance is achieved at $C \approx 0.046$, with validation accuracy of approximately 0.9855.

C. Random Forest

Random forest specific stuff

Once the models were evaluated, confusion matrices, ROC curves, and computed AUC scores were computed for each of them. The models themselves were also saved to later be used for inference. This inference was performed with identical dataset preprocessing on the unbalanced dataset. Confusion matrices, ROC curves, and AUC scores were also computed for the inference run, and only one run was performed per model as the results are deterministic.

Throughout both training and analysis, performance statistics were collected. These were RAM usage, CPU time, and wall clock time. Training was done on a desktop computer equipped with a 12th Gen Intel(R) Core(TM) i7-12700K and 32GiB of DDR5 4800MHz RAM, and inference was ran on both the same desktop computer and a Raspberry Pi 4 with a Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz and 4GiB of DDR4 3200MHz RAM. This was done to measure performance under limited hardware.

VI. RESULTS

A. kNN

Training costs are a one-time expense, but they're relevant when considering that models need to be occasionally retrained as attacks evolve and signatures change. As shown in Table I, LR is by far the cheapest to train, using only 233.83 MB of RAM and completing in 42 seconds of wall time. RF is significantly more expensive but still tractable at 116 seconds. kNN is the most costly by a wide margin, consuming over 2 GB of RAM and taking nearly 20 minutes of wall time. A significant portion of the training time for kNN comes from the 5-fold cross-validation: having to compute the distances between 200 000 samples is incredibly slow. For larger datasets and more thorough validation, kNN may be completely unrealistic as an option.

To contextualise Table II: a 1 Gb/s network carrying TCP/UDP traffic generates 5 000 to 50 000 NetFlow records per second. A lightly loaded home or small office router might produce 100 to 1 000 flows/s, while enterprise or data-centre networks can exceed 100 000 flows/s.

kNN is a clear bottleneck. On the desktop hardware it achieves only 1 409 flows/s which is equivalent to a home router under a medium load, and on the Raspberry Pi 4 it achieves 142 flows/s which is catastrophically low and would result in essentially any networy traffic overwhelming the model, causing inference to fail - let alone a DDoS attack.

TABLE II
RESOURCE USAGE FOR MODEL INFERENCE ACROSS HARDWARE, EVALUATED ON 200 000 SAMPLES.

Model	Hardware	RAM Δ (MB)	Wall Time (s)	Through-put (flows/s)	Latency (ms/flow)
kNN	Desktop	60.95	141.90	1409	0.710
kNN	Raspberry Pi 4	290.61	1408.23	142	7.041
RF	Desktop	6.48	7.95	25145	0.040
RF	Raspberry Pi 4	64.94	65.15	3070	0.326
LR	Desktop	36.73	7.42	26943	0.037
LR	Raspberry Pi 4	68.63	30.79	6497	0.154

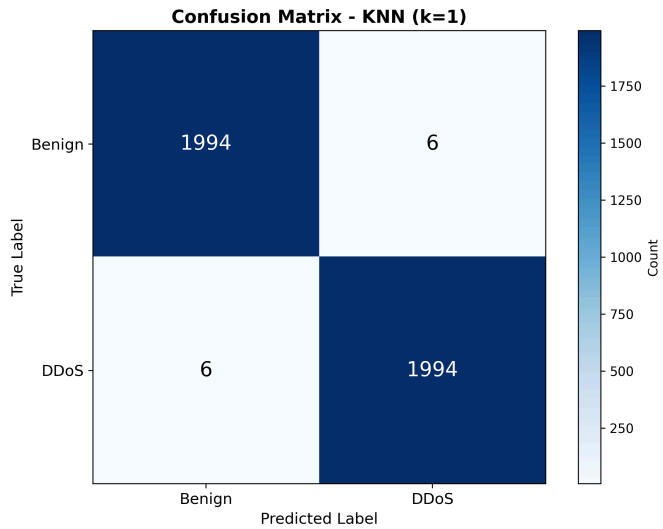

Fig. 3. Confusion matrix for kNN during training, seeing very high recall.

This makes kNN the least efficient model of the three. The low amount of traffic that it can handle disqualifies it from being useful in any realistic scenario, and its low recall on DDoS traffic makes it evident that it has extremely limited usefulness on what traffic it can make decisions on. Further work is needed to refine the kNN model: reducing the feature space, as explored in [2], is a promising direction to take as their approach achieved competitive performance with only four features which would substantially reduce runtime. This is very relevant for IoT deployment, where resource constraints can be even more severe than on the Raspberry Pi 4; [3] highlight that this kind of environment would require that further optimisation is needed.

Prior to finalising the model training, hyperparameters were evaluated from 1 to 50. This hyperparameter k represents the number of neighbours that kNN uses. As shown in Fig. 1, both training and validation accuracy are the highest for $k = 1$ ($\approx 99.7\%$ for both), and decline as k increases. This is a worrisome result that looks like extreme overfitting on the training set, however both LR and RF had similar results. Therefore, it's possible that the balanced dataset represents a genuinely simple problem to solve and that DDoS traffic is

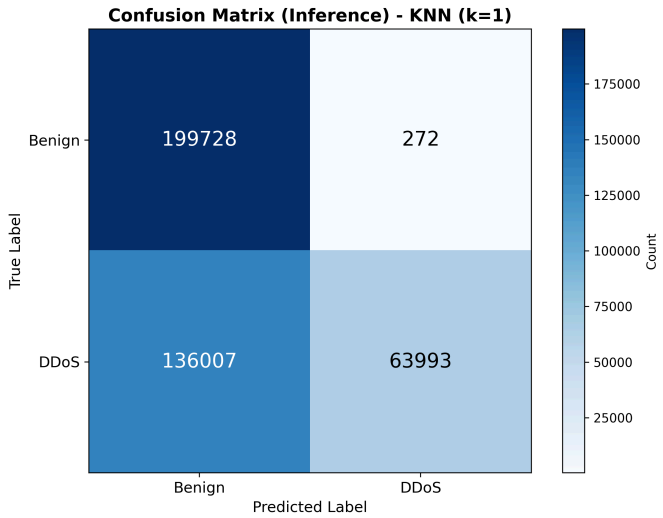

Fig. 4. Confusion matrix for kNN during inference, seeing a high false negative rate.

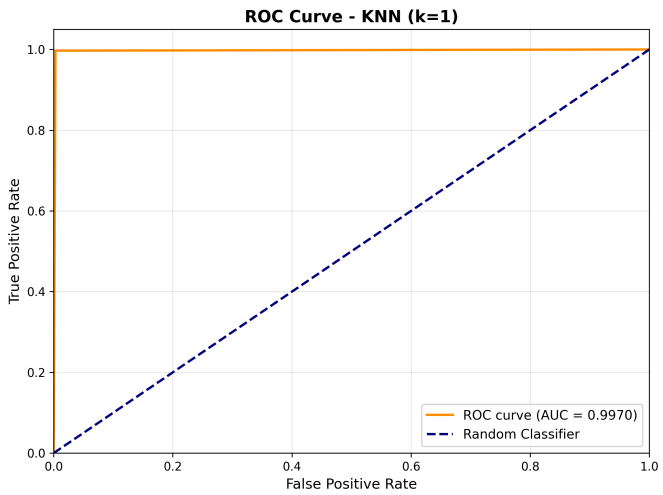

Fig. 5. ROC curve for kNN on the balanced test set, with a AUC of 0.9970.

distinct enough from benign traffic for it to be trivial to tell them apart. Unfortunately, the collapse in kNN's recall for the unbalanced dataset signifies definitively that overfitting did occur, as it to perform on new data and has therefore cannot sufficiently classify realistic, unbalanced traffic.

Evaluated against the balanced dataset, the kNN model with a hyperparameter of $k = 1$ achievees near perfect recall. The training/validation confusion matrix (Fig. 3) shows shows 3 988 correct classifications and just 12 misclassifications, yielding an accuracy of 99.7% and equal recall of 99.7% for both classes. The corresponding ROC curve (Fig. 5) achieves an AUC of 0.9970, confirming that the model can almost perfectly separate DDoS and benign traffic when class distribution is balanced.

To assess the viability of kNN in a real-world scenario, inference was performed on a dataset with 80% benign traffic and 20% DDoS, with 200 000 samples. This is a much more difficult scenario, as there is significantly less DDoS traffic to train on. The inference confusion matrix (Fig. 4) shows a degradation in DDoS recall with this unbalanced dataset. Of 200 000 samples, only 63 993 were correctly identified, yielding a DDoS recall of 32.0%, a huge collapse when compared to the balanced dataset. This disparity comes from the distribution of data in our feature space - with a decision boundary created for balanced probability, a large amount of DDoS traffic samples are in regions dominated by benign traffic samples, creating many false negatives. The near-zero false positive rate confirms that the model can identify DDoS traffic that's representative, but fails when it's more similar to benign traffic.

LR and RF saturate 1 Gb/s network, but there's also 2.5, 5, and 10 Gb/s networks iirc. flow/s increases linearly so figure out what we can reach, where it's used, and how useful that is. Latency is no issue for any of our models, web requests are 100ms and games are like 10ms best case.

B. Logistic Regression

The results show when C is too small it leads to underfitting, reducing both training and validation accuracy. When increasing C, the performance improves until it reaches $C \approx 0.046$. From then on, increasing C any further has practically no effect to the performance. The overfitting gap remains almost zero across all values, indicating that the model generalises well and does not overfit.

This suggests that the data might be close to linearly seperable under our current feature representation. At this point the model is also flexible enough to capture the structure in the data, which is why even more additional capacity does not improve its performance. Once As soon as a certain level of flexibility is reached, adding more capacity does not improve the performance. Thus we can categorize it in a low-bias, low-variance regime, where both underfitting and overfitting effects are minimal.

While regularisation has little influence, the classification does have a strong impact on how the model behaves. Using the best validation model, we evaluate thresholds between 0.1 and 0.9.

Increasing the threshold makes the classifier more conservative. This means that fewer flows are classified as DDoS, which reduces the FPR and increases precision, but lowers recall. In reverse, the advantage lowering the threshold is that it maximizes the number of detections, but the trade-off is that it also increases false positives.

For example, at threshold 0.5, the model achieves FPR ≈ 0.028 and recall ≈ 0.9995. If the threshold is increased to 0.9, it reduces the FPR to approximately 0.015, however the recall drops to around 0.9588. This clearly shows the trade-off between how sensitive the detection is and the false alarm rate.

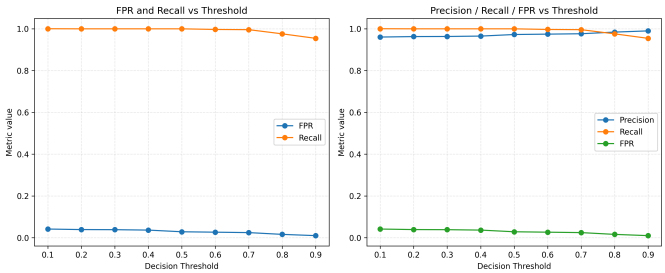

Fig. 6. Threshold sweep for logistic regression. Increasing the decision threshold reduces false positives and increases precision, but lowers recall.

These results show that model performance doesn't just depend on a single fixed setting, but rather on the chosen threshold, which controls the balance between detecting actual attacks and avoiding false positives. In contrast, regularisation has little effect once it reaches moderate values.

Another advantage of logistic regression is that it's easy to interpret, as the learned coefficients overtly show how each feature directly influences the prediction. Positive coefficients push predictions toward the DDoS class, while negative coefficients support the benign class.

The most important DDoS-indicating features include:

- `Init Bwd Win Bytes`
- `SYN Flag Count`
- `Protocol_6`
- `Down/Up Ratio`

These features align with known attack characteristics, such as abnormal TCP flag usage and protocol-level patterns typical of DDoS traffic.

The most important benign-indicating features include:

- `Fwd Segment Size Min`
- `Fwd Packet Length Max`
- `Protocol_17`
- `PSH Flag Count`

These features correspond to more stable and structured flow behaviour associated with legitimate traffic.

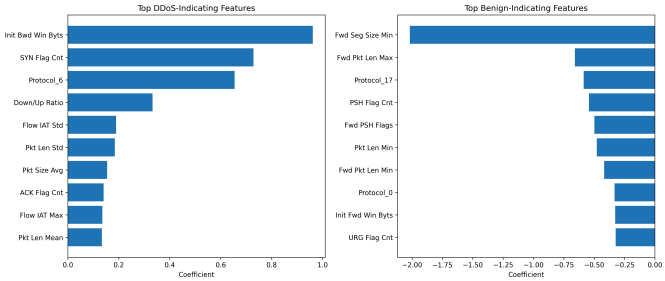

Fig. 7. Most influential logistic regression coefficients. Positive values support the DDoS class, while negative values support the benign class.

It is noteworthy, that the most important features match commonly known DDoS characteristics, suggesting that the model does not rely on random correlations, but is in fact learning meaningful and real patterns. This is useful in security settings, where it is relevant to understand why a prediction is made.

Using the best validation setup, logistic regression performs well on the test-set, reaching an accuracy of 0.9855 and a ROC-AUC of approximately 0.996.

Class-wise performance reveals a slight asymmetry:

- Benign recall: 0.9715
- DDoS recall: 0.9995

This indicates that the model almost never misses attack traffic, but sometimes misclassifies benign flows as attacks. The confusion matrix confirms that most errors correspond to false positives rather than false negatives.

Prioritising catching attacks over avoiding false alarms leans into a conservative detection strategy. While this may be the safer option for security settings, it also leads to unnecessary blocking of legitimate traffic. Therefore selecting an appropriate decision threshold is the most important factor when balancing the trade-off between detection perfoamcne and acceptable false positive rates.

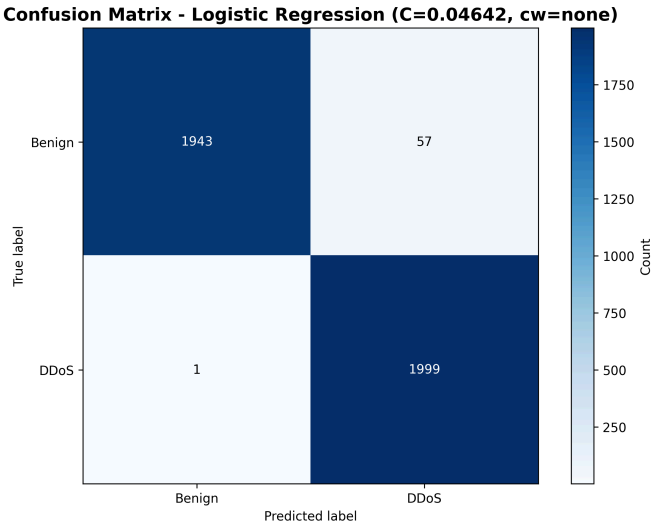

Fig. 8. Confusion matrix on the test set. Errors are dominated by false positives rather than missed attacks.

Cross-validation yields a mean accuracy of (insert CV mean) with standard deviation (insert CV std), indicating that performance is stable across different data splits and not driven by a particular partition of the data.

A limitation of this evaluation is that the dataset is explicitly balanced and heavily preprocessed, which may simplify the classification task. In real-world scenarios, class imbalance and noisier feature distributions could reduce performance and increase the importance of threshold tuning.

C. *Random Forest*

VII. Conclusion

kNN proves unsuitable for practical DDoS detection as implemented in this report. While it does achieve near-perfect accuracy of 99.7% on the balanced dataset, this does not generalise to realistic conditions where traffic is unbalanced: DDoS recall collapses to 32.0%, confirming that overfitting occurred. Its inference throughput of 1 409 flows/s on desktop

hardware and just 142 flows/s on the Raspberry Pi 4 further disqualify it from being used in any real scenario, as even a lightly loaded home router would overwhelm it. The primary issue is the cost of distance computation in the high-dimensional feature space that was used. With 79 features and 200 000 samples, both training and inference are prohibitively slow. Future work should investigate reducing the feature space as in [2], or applying class-weighted voting to address the imbalance sensitivity, rather than letting the majority class crowd out the minority class.

Logistic regression provides a strong, stable, and interpretable baseline for DDoS detection on this dataset.

First, performance is largely insensitive to the regularisation parameter once the model is no longer under-constrained, and no significant overfitting is observed. Second, the classification threshold has a much larger effect on the trade-off between false positives and recall, making it the primary tuning mechanism for deployment. Third, coefficient analysis shows that the model relies on meaningful traffic features such as TCP flags, protocol indicators, and flow-level statistics.

Taken together, these results suggest that the dataset is close to linearly separable and that logistic regression is sufficient to capture most of the available signal, while offering the advantages of simplicity, computational efficiency, and interpretability.

References

[1] S. K. S. P. Y. e. a. Rajendran R., "Detection of DoS attacks in cloud networks using intelligent rule based classification system.," *Cluster Computing*, vol. 22, no. ?_number, pp. 423–434, Feb. 2019, doi: 10.1007/s10586-018-2181-4.

[2] S. Dong and M. Sarem, "DDoS Attack Detection Method Based on Improved KNN With the Degree of DDoS Attack in Software-Defined Networks," *IEEE Access*, vol. 8, no. , pp. 5039–5048, 2020, doi: 10.1109/ACCESS.2019.2963077.

[3] A. I. Gide and A. A. M. zu, "A Real-Time Instrusion Detection System for DoS/DDoS Attack Classification in IoT Networks Using KNN-Neural Network Hybrid Technique," *Babylonian Journal of Internet of Things*, vol. 2024, no. , pp. 60–69, 2024, doi: 10.58496/BJIoT/2024/008.

[4] E. Boadi, "Comparative Analysis of Machine Learning Classifiers using the NSL-KDD Dataset," *ResearchGate*, 2025.

[5] S. Chalichalamala, N. G., and K. Ramani, "Logistic Regression Ensemble Classifier for Intrusion Detection System in Internet of Things," *Sensors*, 2023, doi: 10.3390/s23239584.

[6] B. Kolukisa, B. K. Dedeturk, H. Hacilar, and V. C. Gungor, "An efficient network intrusion detection approach based on logistic regression model and parallel artificial bee colony algorithm," *Computer Standards & Interfaces*, vol. 89, 2024, doi: 10.1016/j.csi.2023.103808.

[7] A. S. C. Clough S. Power, "Project Codebase on Github." [Online]. Available: https://github.com/Flootersmm/ml_project